The µOS++ test catalogue

Every test the harness registers, per architecture, platform and board: what each test does, whether it runs the SMP scheduler or a single core, how it is run (QEMU, host or the real board) and, for the board runs, which debug probe.

The sources of truth are the code, not other documents:

What	Where
which cases exist	`tests/platforms/<platform>/CMakeLists.txt`; for port platforms, the directories under `<port>/test/<board>/` (globbed at configure time)
per-test knobs (core count, HWD_ONLY, sources)	`<port>/test/<board>/tests.cmake`
board facts (CPU, core count, QEMU linker)	`<port>/test/boards/<board>/board.cmake`
probe	`<port>/test/boards/<board>/openocd*.cfg` and `hw.sh`; for the upstream platforms, the `openocd` command in `tests/platforms/<platform>/CMakeLists.txt`
xpm actions	`tests/package.json`

The ports are the sibling repositories `micro-os-plus-iii-aarch32`, `-aarch64`, `-cortexm` and `-posix-arch` (checked with or without `.git` suffix); the drivers and SoC support of the former `-devices` repository are inside each port (`drivers/`, `soc/<chip>/`) since 2026-10-06; the unified SMP kernel and harness root is `micro-os-plus-iii` (branch `smp`), with tests driven from `package.json` at root or under `tests/`.

1. Legend

Variant (the suffix of the CTest name):

Suffix	Runs on	Runner
`-qemu`	QEMU	`test_smpl/run-qemu.sh`, or QEMU directly (Cortex-M)
`-host`	the Linux host (POSIX port)	`test_smpl/run-host.sh`, or the executable directly
`-hwd`	the real board, through OpenOCD	the board's `hw.sh` (then `test_smpl/run-hw.sh` on the Pis)
`-test` (no variant)	harness-only platforms: QEMU, or the board through OpenOCD	the command in the platform's `CMakeLists.txt`

`-hwd` cases carry the `hwd` label and are excluded from `xpm run test` (`ctest -LE hwd`). They are run one at a time, one per power cycle, with OpenOCD only (no GDB); the verdict is the `RESULT: PASS|FAIL` line, read from semihosting and mirrored on the UART.

Scheduling (the "Mode" column below):

Mode	Meaning
SMP n	kernel built with `OS_USE_SMP_SCHEDULER`, `OS_NCPU = n`, all n cores scheduling
SMP n, 1 active	SMP build, but the test keeps the secondaries parked
SMP ×1	SMP scheduler code paths at `OS_NCPU = 1` (single-CPU board)

Mode	Meaning
single	the kernel's non-SMP branch (`OS_USE_SMP_SCHEDULER` undefined), one core
bare	no kernel at all: a bare-metal probe

2. Platforms at a glance

Platform (xpm config prefix)	Port	Board / machine	CPU	Cores	Variants	Probe
`aarch32-rpi-zero-2w`	aarch32	Raspberry Pi Zero 2 W (BCM2837)	4× Cortex-A53, AArch32	4	qemu, hwd	J-Link (default) or Olimex
`aarch32-rpi3b`	aarch32	Raspberry Pi 3 B (BCM2837)	4× Cortex-A53, AArch32	4	qemu, hwd	J-Link (default) or Olimex
`aarch32-luckfox-lyra`	aarch32	Luckfox Lyra B (RK3506)	3× Cortex-A7 (+ a Cortex-M0 not scheduled)	3	hwd only	WCH-Link, CMSIS-DAP
`aarch64-rpi-zero-2w`	aarch64	Raspberry Pi Zero 2 W (BCM2837)	4× Cortex-A53, ARMv8-A	4	qemu, hwd	J-Link (default) or Olimex
`aarch64-rpi3b`	aarch64	Raspberry Pi 3 B (BCM2837)	4× Cortex-A53, ARMv8-A	4	qemu, hwd	J-Link (default) or Olimex
`cortexm-pico2`	cortexm	Raspberry Pi Pico 2 (RP2350)	2× Cortex-M33	2	qemu (6 tests), hwd, ram (4 tests)	any CMSIS-DAP
`cortexm-pico2-pizero`	cortexm	Pi-Zero RP2350B, 16 MB flash	2× Cortex-M33	2	hwd only	XV-Link CMSIS-DAP
`cortexm-pico2-rp2350b-psram`	cortexm	WeAct RP2350B, 16 MB flash + 8 MB PSRAM	2× Cortex-M33	2	qemu (3 tests), hwd	CMSIS-DAP `c251:f001`
`cortexm-pico`	cortexm	Raspberry Pi Pico (RP2040)	2× Cortex-M0+	2	qemu (6 tests), hwd, ram (4 tests)	JTAGprobe CMSIS-DAP `2e8a:000c`
`cortexm-nucleof411`	cortexm	ST Nucleo-F411RE	1× Cortex-M4F	1	qemu (netduinoplus2, 4 tests), hwd	on-board ST-Link v2.1
`cortexm-weactf411`	cortexm	WeAct Studio F411CE	1× Cortex-M4F	1	qemu (netduinoplus2, 5 tests), hwd	DAPLink CMSIS-DAP (or WCH-Link)
`cortexm-weactf412`	cortexm	WeAct Studio F412RE	1× Cortex-M4F	1	hwd only (SRAM 256K > QEMU 128K)	ST-Link
`native`	posix-arch	the Linux host	host threads as CPUs	4 (`-DNCPU`)	host	—
`2xcortex-m33`	cortexm	QEMU `mps2-an521`	2× Cortex-M33	2	QEMU	—
`pico2-1cpu`	cortexm	QEMU `mps2-an505`	1× Cortex-M33	1	QEMU	—

Platform (xpm config prefix)	Port	Board / machine	CPU	Cores	Variants	Probe
`qemu-cortex-m0`	cortexm	QEMU `mps2-an385`	Cortex-M3 (the platform passes `--cpu cortex-m3`, not an M0)	1	QEMU	—
`qemu-cortex-m3`	cortexm	QEMU `mps2-an385`	Cortex-M3	1	QEMU	—
`qemu-cortex-m4f`	cortexm	QEMU `mps2-an386`	Cortex-M4	1	QEMU	—
`qemu-cortex-m7f`	cortexm	QEMU `mps2-an500`	Cortex-M7	1	QEMU	—
`nucleo-f411re`	harness (upstream)	ST Nucleo-F411RE	Cortex-M4F	1	board	ST-Link (`interface/stlink-dap.cfg`)
`nucleo-f767zi`	harness (upstream)	ST Nucleo-F767ZI	Cortex-M7	1	board	ST-Link (`interface/stlink-dap.cfg`)
`nucleo-h743zi`	harness (upstream)	ST Nucleo-H743ZI	Cortex-M7	1	board	ST-Link (`interface/stlink-dap.cfg`)
`raspberrypi-pico`	harness (upstream)	Raspberry Pi Pico (RP2040)	Cortex-M0+	1	board	CMSIS-DAP (`interface/cmsis-dap.cfg`)

Every platform has a `-cmake-gcc-debug` and a `-cmake-gcc-release` configuration; `native` also has `-cmake-sys-*`, `-cmake-gccNN-*` and `-cmake-clangNN-*`. Only these configurations carry per-test `test-<app>-<variant>` actions: the `cortexm-*`, `aarch32-*`, `aarch64-*`, `qemu-cortex-*`, `2xcortex-m33`, `pico2-1cpu`, `native-cmake-gcc-*` and `native-cmake-sys-*` ones (the release configurations inherit them from debug).

The QEMU machine for the Pis is `raspi3b -smp 4`; the AArch32 images are started through a small AArch64 boot shim (`shim8.img`, image at `0x10000`). The Cortex-M `-qemu` images of `cortexm-pico2` and `cortexm-pico2-rp2350b-psram` run on `mps2-an500 -cpu cortex-m7`; those of `cortexm-pico` on `mps2-an385 -cpu cortex-m3` (the generic M0 core).

The four `qemu-cortex-*` platforms link the local Cortex-M port's generic single-core QEMU cores (`micro-os-plus::cortexm-qemu-m0/-m3/-m4f/-m7`) and the generic device in `tests/device-qemu-cortexm`, and run the SMP kernel single-core (`OS_NCPU=1`, no `OS_USE_SMP_SCHEDULER`). Each registers three suites: `<platform>-rtos-apis-test`, `-mutex-stress-test` and `-cmsis-os-validator-test`. (`2xcortex-m33` and `pico2-1cpu` likewise link the local generic M33 core, `micro-os-plus::cortexm-qemu-m33`.)

3. Probes

Board	Probe	USB id	Transport, speed	Config
Pi Zero 2 W, Pi 3 B (aarch32 and aarch64)	SEGGER J-Link — default	—	JTAG	`test/boards/rpi-zero-2w/openocd-jlink-rpi3.cfg`

Board	Probe	USB id	Transport, speed	Config
Pi Zero 2 W, Pi 3 B, with `PROBE=olimex`	Olimex ARM-USB-OCD (FTDI)	—	JTAG, 1000 kHz	`test/boards/rpi-zero-2w/openocd-olimex.cfg`
Pi Zero 2 W, Pi 3 B (aarch32 and aarch64), with `PROBE=cmsis-dap`	any CMSIS-DAP (Raspberry Pi Debug Probe, a Pico with the debugprobe firmware)	`2e8a:000c`	JTAG (the target config's clock: 4000 kHz aarch64, 1000 kHz aarch32)	`test/boards/rpi-zero-2w/openocd-cmsisdap-rpi3.cfg`
Luckfox Lyra B	WCH-Link in ARM mode (CMSIS-DAP)	`1a86:8011`	SWD, 4000 kHz	`test/boards/luckfox-lyra/openocd.cfg`
Pico 2	any CMSIS-DAP (auto-selected, no vid_pid pinned)	—	SWD, 1000 kHz	`test/boards/pico2/openocd.cfg`
Pi-Zero RP2350B	XV-Link CMSIS-DAP	`0416:5951`	SWD, 2000 kHz	`test/boards/pico2-pizero/openocd.cfg`
WeAct RP2350B + PSRAM	CMSIS-DAP	`c251:f001`	SWD, 1000 kHz	`test/boards/pico2-rp2350b-psram/openocd.cfg`
Pico (RP2040)	JTAGprobe (JTAGprobe_smp firmware on a second Pico), CMSIS-DAP v2	`2e8a:000c`	SWD, 15000 kHz (`UOS_HW_ADAPTER_KHZ`)	`test/boards/pico/openocd.cfg`
Nucleo-F411RE	on-board ST-Link v2.1	—	SWD, `connect_assert_srst`	`test/boards/nucleof411/openocd.cfg`
WeAct F411CE	DAPLink CMSIS-DAP (override: WCH-Link `1a86:8011` through `DAP_VID_PID`)	`0d28:0204`	SWD, 1000 kHz (`ADAPTER_KHZ`)	`test/boards/weactf411/openocd.cfg`
WeAct F412RE	ST-Link V2 / V2-1 / V3	by class	SWD (`ADAPTER_KHZ`)	`test/boards/weactf412/openocd.cfg`

The Pi 3 B `hw.sh` is a wrapper that `exec`s the Zero 2 W's, so both Pis take the same `PROBE` switch. The paths above are relative to the port repository (`aarch32` or `aarch64` for the Pis and the Lyra, `cortexm` for the rest).

4. The harness suites (portable)

Their sources live in `tests/sources/<suite>/`. They have no `main()`: the entry is `os_main()`, and its return code is the verdict. On the port boards a per-board wrapper wraps `os_main` (`-Wl,--wrap=os_main`), prints the `RESULT:` line and stops the run. It is each suite's `harness-suite.cpp` on the Pi ports, and one `test/<board>/harness-suite.hpp` per board, included by the suites' `harness-suite.cpp`, on cortexm.

Suite	What it does
`rtos-apis`	Exercises the µOS++ C++ API, C API, ISO (`std::`) API, CMSIS-RTOS v1 wrapper and the POSIX I/O layer, plus a Chan FatFs test.
`mutex-stress`	Several threads take one mutex at random intervals; the test checks that every thread got it and the distribution is sane.
`fp-switch`	

Suite	What it does
	Six unpinned threads at three priorities each load their own pattern into s0-s31 and the FPSCR flags, spin long enough to be preempted (and, on SMP, moved to the other core), and check the registers read back unchanged. Needs an FPU; without one it passes with nothing to test.
`cmsis-os-validator`	The Arm CMSIS-RTOS v1 validator (60 test cases: threads, timers, signals, semaphores, mutexes, memory pools, message and mail queues, with interrupt-context cases). On a Cortex-M the interrupt is NVIC IRQ 0; see below for the other cores.
`blinky`	Blinks the LED. Enabled only on `nucleo-f411re` (`platforms/nucleo-f411re/cmake/definitions.cmake`), where it is the case `blinky-test`.
`instrumentation`	SEGGER SystemView use cases. Enabled only on `nucleo-f411re`, which builds `instrumentation-test` and `rtos-apis-instrumentation-test` but registers no CTest case for them: they are run from SEGGER Ozone with a J-Link (see `tests/sources/instrumentation/README.md`).

Where each suite runs:

Platform	`rtos-apis`	`mutex-stress`	`cmsis-os-validator`	Mode
`aarch32-rpi-zero-2w`, `aarch32-rpi3b`, `aarch64-rpi-zero-2w`, `aarch64-rpi3b`	qemu, hwd	qemu, hwd	qemu, hwd	SMP 4
`aarch32-luckfox-lyra`	—	hwd (`mutex-stress-test`)	—	SMP 3
`cortexm-pico2`	qemu, hwd, ram	qemu, hwd, ram	qemu, hwd, ram	SMP 2 on hwd; single on qemu & ram (SRAM load)
`cortexm-pico`	qemu, hwd, ram	qemu, hwd, ram	qemu, hwd, ram	as `cortexm-pico2`
`cortexm-nucleof411`, `cortexm-weactf411`	qemu, hwd	qemu, hwd	qemu, hwd	SMP ×1
`cortexm-weactf412`	hwd	hwd	hwd	SMP ×1
`native`	host (`rtos-apis`, single; `smp-rtos-apis`, SMP 4)	host (`mutex-stress`, single; `smp-mutex-stress`, SMP 4)	host	see §8
`2xcortex-m33`	QEMU	QEMU	QEMU	SMP 2
`pico2-1cpu`	QEMU	QEMU	QEMU	single
`qemu-cortex-m0/m3/m4f/m7f`	QEMU	QEMU	QEMU	single (local cortexm port's generic QEMU cores, `OS_NCPU=1`)
`nucleo-*`, `raspberrypi-pico`	yes	yes	yes	single (upstream); `nucleo-f411re` adds `blinky-test`

`fp-switch` runs where an M33 FPU does: `cortexm-pico2` (qemu, hwd), `2xcortex-m33` (QEMU, SMP 2) and `pico2-1cpu` (QEMU, single). On `cortexm-pico` (Cortex-M0+, no FPU) it builds and passes with nothing to test.

On `cortexm-pico2` the `-qemu` image of a suite does not run the board at all: QEMU has no RP2350, so the image links the generic single-core Cortex-M QEMU core in its place, and the port's builder forces `NCPU` to 1 for it (`<cortexm>/test/CMakeLists.txt`).

The validator is CMSIS-RTOS v1, which assumes one CPU. On the SMP platforms the kernel's wrapper pins every thread `osThreadCreate()` makes to core 0 (`src/rtos/os-c-wrapper.cpp`). On the Pi ports the board's `harness-suite.cpp` pins the main thread too; the pico2 wrapper (`test/pico2/harness-suite.hpp`) does not, so there the validator's main thread keeps the default affinity while every thread it creates is on core 0. On the BCM2837 (both Pi ports) its NVIC IRQ 0 is emulated with the local Mailbox 1 interrupt, and the Cortex-M DWT cycle counter it probes is answered by a RAM page that says "no cycle counter". Both exist only in the validator's image (`UOS_CMSIS_OS_VALIDATOR`). On `native`, the validator xpack's own `SIGUSR1`/`SIGALRM` shims stand in for the NVIC.

5. AArch32 and AArch64 — Raspberry Pi Zero 2 W and Pi 3 B

The same sixteen test directories exist on all four (port, board) pairs: `<port>/test/rpi-zero-2w/` and `<port>/test/rpi3b/`. The Pi 3 B builds use the Zero 2 W's board sources with `BOARD_RPI3B`. Every test is built twice, `-qemu` and `-hwd`, except as noted.

Test	What it does	Mode
`smp_test0`	Phase-1 bring-up: PL011 UART, MMU (cacheable, shareable DRAM), the 1 ms generic-timer tick and the scheduler, with one thread printing 10 heartbeats.	SMP 4, 1 active
`smp_test1`	Semaphore ping-pong across cores: pinger on core 0, ponger on core 1, a logger on core 2 blinking the LED and printing the round count and cores.	SMP 4
`smp_test2`	Lock coherency: one worker pinned per core bumps a shared counter under a mutex; the final count must be `OS_NCPU × ITER`.	SMP 4
`smp_test3`	`message_queue` producer (core 0) and consumer (core 1) that tests primes and toggles the LED; a logger prints throughput.	SMP 4
`smp_test4`	Load balancing: CPU-bound workers with no affinity; a reporter prints each worker's per-core histogram.	SMP 4
`smp-mat-test`	Parallel block Gaussian elimination (N = 120, B = 20) across all cores with spin barriers, a thread-pool demo, and a classical LU solve as ground truth; compares and times both.	SMP 4
`smp-mat-sdcard-test`	The same solver at N = 200 with the matrices stored on and loaded from the SD card (flatfs on `disk.img` under QEMU; FatFs under `/tests` on the boot card on hardware).	SMP 4
`smp-num-test`	Five prioritised threads: UART ticker, LED blinker, an SD text writer, an FP compute writer and a reader, all sharing `num.txt` under one mutex, paced by a counting semaphore.	SMP 4

Test	What it does	Mode
`smp-pipeline-test`	13 threads (4 producers, 4 compute workers, 2 SD writers, an auditor that checks CRC and sequence, an LED pacer, telemetry) with heavy `yield()`.	SMP 4
`smp-pro-cons-test`	Producer/consumer that uses every kernel object (thread, memory pool, message queue, both semaphores, mutex, condition variable, event flags, timer, sysclock, suspend/resume).	SMP 4
`sd_test`	SD card: under QEMU, `sd::SdCard` + flatfs on a host-seeded `disk.img` (host ↔ device round trip); on hardware, FatFs read/write under `/tests` on the boot card (never formatted).	SMP 4
`usb_test`	DWC2 USB gadget file transfer: a host tool sends files over bulk OUT, stored to the SD card and hex-dumped to the UART and semihosting, with unpinned workers keeping every core busy.	SMP 4
`clock-test`	The kernel's clocks against the BCM2837 System Timer: 5000 ticks of `sysclock` and `hrclock` keep its time; `hrclock.sleep_until (now () + 100 ms)` lasts 100 ms on cores 0 and 1, each woken promptly after core 0's tick. Guards the 2026-10-22 clock fixes.	SMP 4
`rtos-apis`, `mutex-stress`, `cmsis-os-validator`	The harness suites (§4).	SMP 4

Exception: `usb_test` has **no `-hwd` case on the Pi 3 B**, only `-qemu`. The Pi 3 B's USB ports sit behind the LAN9514 hub, so the DWC2 can only be a host.

Case counts: Zero 2 W 32 (16 × 2), Pi 3 B 31.

6. AArch32 — Luckfox Lyra B (RK3506)

Hardware only: the board declares no QEMU linker script. 20 port tests plus one harness suite, all `-hwd`, all SMP 3.

Test	What it does
`smp_test0` … `smp_test4`	The same bring-up, ping-pong, lock-coherency, message-queue and load-balancing tests as on the Pis (§5), on three cores.
`smp-mat-test`, `smp-mat-sdcard-test`, `smp-num-test`, `smp-pipeline-test`, `smp-pro-cons-test`, `sd_test`	As on the Pis (§5), on the Lyra's SD card controller (DW-MMC).
`smp_test5`	Three pinned threads: an SD card thread on core 0 (FatFs mount, formatting FAT32 if none), an LED blinker on core 1 and a logger on core 2.
`smp_test6`	USB-to-UART logger on three cores: a DWC2 gadget task, a heartbeat logger and an LED task; driven from the host with `usb_test_host`.
`smp_test7`	

Test	What it does
	USB vendor gadget receives a file, the SD task writes it as `received.dat`, then asks on the UART whether to delete it; the LED reflects the answer.
`smp_test_int`	First external interrupt: UART1 RX on the GIC (ID 99), with an ISR echo and a TX line lock shared with the ISR.
`smp_test_int2`	Interrupt-driven TX through a THRE ring, and the UART1 interrupt routed to core 1 via `GICD_ITARGETSR`.
`smp_test_int3`	PC keyboard → board: USB OTG1 as an interrupt-driven vendor gadget on core 0 (GIC ID discovered at runtime), UART1 interrupt-driven on core 1. Needs the host tool `kbd_forward`.
`smp_test_int4`	Keystroke logger with three IRQs on three cores: USB (core 0) → ring → `keys.log` on the card, written with the DW-MMC data-transfer-over IRQ (core 2); UART on core 1.
`smp_test_int5`	AMP doorbell: the RK3506's Cortex-M0 fires Mailbox interrupts every ~40 ms to A7 cores 1 and 2, which report to core 0 over a shared ring.
`clock-test`	The kernel's clocks against the generic timer's counter read directly (CNTPCT, 24 MHz; the RK3506 has no BCM System Timer): 5000 ticks keep its time, `hrclock.sleep_until (now () + 100 ms)` lasts 100 ms on cores 0 and 1. Its console is the UART alone, sent through semihosting once at the end.
`mutex-stress-test`	The harness suite, built by the platform itself (not a port test directory).

Every Lyra test ends with a `RESULT:` line and a semihosting exit. The four USB tests (`smp_test6`, `smp_test7`, `smp_test_int3`, `smp_test_int4`) wait up to 120 s for the host to configure the gadget, then fail; they are built `UOS_CONSOLE_UART_ONLY`, so their console does not mirror every line through semihosting (each such trap halts the core, and during USB traffic that lost the WCH-Link's debug link). Their host tools are C/C++ sources in the test's own `host/` directory: `usb_test_host` for `smp_test6`/`smp_test7`, `kbd_forward` for `smp_test_int3`/`smp_test_int4`.

7. Cortex-M

7.1 RP2350 boards

Three boards on the same silicon (2× Cortex-M33): `pico2`, `pico2-pizero` and `pico2-rp2350b-psram`. On the single-core tests `OS_NCPU` is 1 (`board_test_ncpu`); the "bare" tests link no kernel (`BOARD_TEST_NO_KERNEL`).

Test	What it does	Mode	pico2	pizero	rp2350b-psram
`exc-test`	Bare exception-entry probe: SysTick with MSP, then PendSV with PSP.	bare	hwd	hwd	hwd
`smp-test0`	Bare boot chain and dual-core launch: C++ constructors, 150 MHz clocks, UART; core 1 blinks the LED.	bare, 2 cores	hwd	hwd	hwd
`smp-test1`	Single-core RTOS bring-up: three threads, sysclock, mutex,	single	qemu, hwd	hwd	qemu, hwd

Test	What it does	Mode	pico2	pizero	rp2350b-psram
	counting semaphore, LED heartbeat.				
`sc-test-ko`	Single-core kernel-object test: ten self-checking sub-tests (mutex, semaphores, queues, event flags, timers, …), controller and helper on core 0.	single	qemu, hwd	hwd	qemu, hwd
`smp-test-ko`	The SMP twin of `sc-test-ko`: controller pinned to core 0, helper to core 1, so every hand-off is cross-core.	SMP 2	hwd	hwd	hwd
`smp-test2`	Dual-core SMP: per-core PendSV and SysTick, affinity (one worker per core), SIO spinlock-backed kernel lock.	SMP 2	hwd	hwd	hwd
`smp-test3`	Cross-core IPI through the SIO FIFO: semaphore ping-pong with the one-way wake latency measured on TIMER0.	SMP 2	hwd	hwd	hwd
`smp-test4`	Kernel objects under dual-core contention: seven round-trip sub-tests between a controller (core 0) and a worker (core 1).	SMP 2	hwd	hwd	hwd
`smp-test5`	First peripheral interrupt: UART0 TX IRQ serviced on core 1, draining a ring filled by core 0.	SMP 2	hwd	hwd	hwd
`clock-test`	The kernel's clocks against TIMER0 on both cores: sysclock and hrclock keep its time, `hrclock.sleep_until (now () + 100 ms)` lasts 100 ms. Guards the 2026-10-22 fixes (hrclock's units and origin, the tick caught up from TIMER0).	SMP 2	hwd	—	—
`smp-mat-test`	Parallel block solver (N = 100, B = 50) on two cores. On `pico2-rp2350b-psram` its data lives in PSRAM.	SMP 2	hwd	hwd	hwd
	USB CDC-ACM virtual serial port: echoes what	SMP 2	hwd	hwd	—

Test	What it does	Mode	pico2	pizero	rp2350b-psram
`smp-test-usb-cdc-acm`	the host types, mirrored on the UART.				
`smp-test-usb-hid`	USB HID keyboard forwarding; keystrokes printed on the UART.	SMP 2	hwd	hwd	—
`psram-exec`	Bare SRAM supervisor that copies two apps into PSRAM and switches between them on a key press.	bare	—	hwd	—
`psram-mat-test-250`	`smp-mat-test` with PSRAM brought up by the app itself, at 250 MHz, from an SRAM-boot linker script.	SMP 2	—	hwd	—
`smp-test-nested`	Nested interrupts on both cores (SysTick, UART0, TIMER0 IRQ 2/3 at different priorities) with thread migration and a lock-free trace. Passes once both workers have run 20 beats and all three nesting interrupts have fired.	SMP 2	—	—	hwd
`smp-test-nested-clock`	`smp-test-nested` plus a clock tree driven from a potentiometer (ADC) and the on-die temperature sensor; data in PSRAM, 250 MHz enabled. **Interactive**: waits for y on the UART (a pot position other than the current clock reboots into it first), then the same 20-beat verdict.	SMP 2	—	—	hwd
`smp-test-nested-clock_200`	The same, without `PICO2_ENABLE_250MHZ` (the clock tree at its default).	SMP 2	—	—	hwd
`smp-test-nested-clock_250`	The same, with 250 MHz enabled.	SMP 2	—	—	hwd
`rtos-apis`, `mutex-stress`, `cmsis-os-validator`	The harness suites (§4).	single	qemu, hwd	—	—
`fp-switch`	The harness suite (§4): six unpinned threads at three priorities keep their own s0–s31 and	single	qemu, hwd	—	qemu, hwd

Test	What it does	Mode	pico2	pizero	rp2350b-psram
	FPSCR flags across preemptive switches.				
`cmsis-os-validator-ram`, `mutex-stress-ram`, `rtos-apis-ram`	RAM-resident harness suites executing entirely from internal SRAM (no flash writes).	single	hwd	—	—
`smp-mat-test-ram`	Parallel block solver executing entirely from internal SRAM (no flash writes).	SMP 2	hwd	—	—

Case counts: `pico2` 26, `pico2-pizero` 14, `pico2-rp2350b-psram` 18.

7.1a RP2040 board

`pico` (Raspberry Pi Pico, RP2040, 2× Cortex-M0+) runs the `pico2` column of the table above, adapted to the RP2040, with no Pico SDK: the same 26 cases (20 `hwd`, 6 `qemu`). The differences: 125 MHz clocks; the RP2040 TIMER, reset bits and IRQs (the SIO FIFO IPI on IRQs 15/16, UART0 on 20, USB on 5); `smp-mat-test` at N = 80, B = 20 to fit 264 KB of SRAM; `exc-test` without UsageFault (ARMv6-M has HardFault only); the USB CDC test lights the onboard LED instead of a WS2812; `fp-switch` has no FPU to test. The `-qemu` images run single-core on `mps2-an385 -cpu cortex-m3`. The probe is the JTAGprobe over SWD at 15 MHz.

All 26 passed on 2026-10-13 (debug): 6/6 QEMU and 20/20 on the board, the two USB tests interactively (Escape then Return in the CDC terminal; Escape through `kbd_forward` for HID). Release: 6/6 QEMU and 18/18 on the board (the two USB tests not run).

7.2 STM32F4 boards (one CPU)

`cortexm-nucleof411` and `cortexm-weactf411` run both on hardware and on QEMU (`-M netduinoplus2 -cpu cortex-m4`). `cortexm-weactf412` is hardware-only. The harness suites build the SMP scheduler at `OS_NCPU = 1`; the board's own tests use the kernel's non-SMP branch.

Test	What it does	Mode	nucleof411	weactf411	weactf412
`mos-test1`	Single-CPU acceptance: producers and consumers over a bounded buffer with plain and recursive mutexes, counting semaphores, sysclock and an LED thread.	single	qemu, hwd	qemu, hwd	hwd
`spi-pipeline`	SPI filesystem pipeline (ported from a Rust example): five threads, eight semaphores, Base64 encode to partition 1, decode and CRC check to partition 2.	single	—	qemu, hwd	—

Test	What it does	Mode	nucleof411	weactf411	weactf412
`uart-test1`	USART1 echo: RX interrupt → bounded buffer → TX via DMA2, with an LED thread.	single	—	—	hwd
`rtos-apis`, `mutex-stress`, `cmsis-os-validator`	The harness suites (§4).	SMP ×1	qemu, hwd	qemu, hwd	hwd

QEMU execution on `netduinoplus2`: - `nucleof411`: 4/4 PASS (`mos-test1`, `rtos-apis`, `mutex-stress`, `cmsis-os-validator`). - `weactf411`: 5/5 PASS (`mos-test1`, `spi-pipeline`, `rtos-apis`, `mutex-stress`, `cmsis-os-validator`). - `weactf412` **hardware-only rationale:** 1. *SRAM limit*: STM32F412RE has 256 KB of SRAM, setting `_estack` at `0x20040000`. QEMU's `netduinoplus2` machine allocates only 128 KB of SRAM (`0x20000000`–`0x20020000`), causing QEMU to fault on startup. 2. *Peripheral support*: `uart-test1` uses DMA2 Stream 7 (`DMA2_Stream7`) for USART1 TX, which is not implemented in the QEMU Netduino Plus 2 model.

Runtime termination fix (`hw_result.hpp`): In `micro-os-plus-iii-cortexm/test/boards/shared/hw_result.hpp`, tests terminate using `std::_Exit(0)` instead of `std::exit(0)`. Normal `std::exit()` invokes `atexit` handlers and static object destructors while RTOS threads are still alive, causing `assert(empty())` assertion failures in `double_list::~double_list()`. `std::_Exit(0)` performs a clean termination and semihosting exit immediately.

8. POSIX — native host

`micro-os-plus-iii-posix-arch/test/native/`. Host threads stand in for CPUs: `NCPU` defaults to 4 (`-DNCPU=1` builds everything single-core). All cases are `-host`.

Test	What it does	Mode
`smp_test0` … `smp_test4`	The Pi tests of the same names (§5), on host threads.	SMP 4 (`smp_test0`: 1 active)
`smp-mat-test`, `smp-num-test`, `smp-pipeline-test`, `smp-pro-cons-test`	As on the Pis; the SD card is flatfs over a host file.	SMP 4
`mutex-stress`	The upstream suite, unchanged.	single
`rtos-apis`	The upstream suite, with the POSIX I/O layer.	single
`smp-mutex-stress`	The SMP leg of `mutex-stress`, with `smp_test2`'s start-up.	SMP 4
`smp-rtos-apis`	The whole `rtos-apis` sequence once per core, from a driver pinned to that core, with the API threads unpinned; fails unless every core did the work.	SMP 4
`flatfs-test`	flatfs on a host-file card: append 64 KiB to a file created empty, then check it, a file beside it and a fresh mount.	SMP 4
`mutex-ceiling-test`	A thread above a robust protect mutex's ceiling gets `EINVAL`; after it is destroyed, lock/unlock at the ceiling must still work (no `EOWNERDEAD`).	SMP 4
`cmsis-os-validator`	The validator (§4), registered by the harness platform itself.	SMP 4, validator on core 0
`clock-test`		SMP 4

Test	What it does	Mode
	`hrclock.sleep_until (now () + 100 ms)` lasts 100 ms of `CLOCK_MONOTONIC`, and `hrclock.now()` is close to its steady count (the 2026-10-22 origin fix: it used to wait the host's uptime).	

17 cases.

9. How to run

From repository root `micro-os-plus-iii` (or `tests/`), for a configuration c such as `aarch64-rpi3b-cmake-gcc-debug`:

```
xpm run install --config C && xpm run prepare --config C && xpm run build --config C
xpm run test --config C                              # every qemu/host case (-LE hwd)
xpm run test-<app>-<variant> --config C              # one case, builds first
PATH="$PWD/build/C/xpacks/.bin:$PATH" ctest --test-dir build/C -N   # list the cases
```

For a `-hwd` case, power-cycle the board first and run exactly one case. On the Pis choose the probe with `PROBE=jlink|olimex|cmsis-dap` (`cmsis-dap` on the Zero 2 W and the Pi 3 B, both ports):

```
cd build/C && PROBE=olimex PATH="$PWD/xpacks/.bin:$PATH" ctest -V -R <platform>-<app>-hwd
```

10. Last verified run

2026-10-06, on the `smp` branches rebuilt commit by commit (xpack-dev-smp.md Part III), latest compilers, every QEMU and host case (`ctest -LE hwd`), debug and release:

Action / platform	Cases passed
`test-native-cmake-sys` (`native`, system gcc and clang)	16/16
`test-cortex-cmake` (`qemu-cortex-m0/m3/m4f/m7f`, gcc and clang)	3/3 in each of the 8 configurations
`aarch32-rpi-zero-2w`, `aarch32-rpi3b`	15/15
`aarch64-rpi-zero-2w`, `aarch64-rpi3b`	15/15
`2xcortex-m33`	4/4
`pico2-1cpu`	4/4
`cortexm-pico2`	6/6
`cortexm-pico2-rp2350b-psram`	3/3
`cortexm-pico` (added later, debug and release)	6/6

The `-hwd` cases and the hardware-only platforms (`cortexm-pico2-pizero`, `cortexm-nucleof411`, `cortexm-weactf411`, `cortexm-weactf412`, `aarch32-luckfox-lyra`) were built but not run. The upstream `nucleo-f411re`, `nucleo-f767zi`, `nucleo-h743zi` and `raspberrypi-pico` platforms fail to configure (the cortexm port does not find the kernel through the xpacks path), as on the reference `smp` branch.